

FAANG Preparation Without Question Chasing

Build transferable coding, design, and communication skills, then tailor the final revision to the actual team and level. Use official hiring guidance for scope, mocks for diagnosis, and a limited resource set for consistent practice. Avoid treating alleged question frequency as a hiring guarantee.

For: Engineers preparing for large product-company interviews who already know a programming language and need a repeatable strategy. FAANG is a convenient grouping, not a uniform interview process.

Suggested pace: 90 days, adapted to the target role | Intermediate SWE

What you will be able to do

- Organize preparation around competency rather than a company's alleged question list.
- Develop coding fluency through mixed practice and independent re-solves.
- Present design trade-offs and truthful behavioral examples with clear evidence.
- Adapt a final revision week to recruiter instructions and demonstrated weak areas.

Build the right mental model

Common skills, different emphasis

Coding fundamentals are reusable, but design depth, specialization, and behavioral expectations vary with level and team. A senior platform role may emphasize operational judgment; an early-career role may spend more time on basic problem solving.

Evidence over memorization

A useful solution includes the problem contract, invariant, implementation, complexity, and tests. Change a constraint and see whether your reasoning still holds. If a memorized answer fails under a small variation, return to the underlying concept.

Separate official scope from anecdotes

Employer preparation pages and recruiter instructions describe the process more reliably than viral interview lists. Candidate anecdotes can inspire practice, but may be outdated, unverifiable, or restricted by confidentiality. Do not claim that a practice example is a confirmed company question.

Study with evidence.

Keep one primary algorithm resource, one design reference, and an error log. Alternate topic-focused learning with mixed problems so recognition is tested without a visible category label. Include regular rest and a realistic schedule around work.

Days 1-21: Diagnose and repair fundamentals

Run coding and project diagnostics. Review arrays, hashing, pointers, windows, stacks, and search. Practice explaining an approach before typing and preserve a short testing segment in each attempt.

Deliverable and check: Three weak-skill targets and a collection of independently corrected failures, not a target solved count.

Days 22-45: Increase transfer

Add trees, graphs, heaps, intervals, backtracking, and DP. Mix patterns and compare multiple approaches. Review networks, OS, and databases through concrete request and failure scenarios.

Deliverable and check: Two weekly timed sets and short explanations of why competing approaches fail or cost more.

Days 46-70: Design and personal evidence

Practice role-relevant architecture, capacity estimates, data contracts, retries, and recovery. Write truthful stories covering ownership, failure, conflict, ambiguity, and collaboration. Rehearse project deep dives with a peer.

Deliverable and check: Several design walkthroughs and stories with verifiable actions and outcomes; no invented metrics or confidential material.

Days 71-90: Simulate the actual loop

Confirm interview structure and tools. Run mocks that resemble the expected rounds, then correct one measurable weakness per session. Reduce new material during the final days and prepare thoughtful role questions.

Deliverable and check: A stable recent scorecard, checked logistics, and a concise revision pack that emphasizes your recurring errors.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Build a company-alignment matrix

You are applying to an infrastructure role and a consumer-product backend role. You cannot double the preparation hours, so you need a shared base and small, intentional specialization tracks.

A worked approach

1. Extract explicit requirements from both job descriptions: coding language, data structures, distributed services, databases, performance, or domain knowledge. Mark requirements that the recruiter confirms and assumptions that remain uncertain.
2. Allocate most time to shared coding and engineering fundamentals. For the infrastructure track, practice failure domains and tail latency; for the product track, practice API contracts, data modeling, permissions, and user-visible consistency.
3. Choose one unseen coding mock and one tailored design per track. Score both using the same correctness and clarity dimensions, then add role-specific depth rather than treating brand names as algorithm categories.

Expected result

A shared core plus two narrow tracks uses time more efficiently than maintaining two unrelated question lists. The matrix should be updated when the actual interview scope changes.

Verify the result

- Does every specialized topic connect to a stated role requirement?
- Are claims about interview format sourced or labeled as assumptions?
- Can you explain each design without relying on a memorized brand architecture?

Extend the exercise

Ask a peer to change one requirement, such as offline operation or stronger consistency. Explain which decisions change and which remain valid; this is a stronger transfer test than redrawing the original diagram.

Make the learning stick.

How many problems should I solve?

There is no universal threshold. Use coverage to find missing patterns, then judge readiness by independent unseen performance, testing, and explanation. Hundreds of copied answers do not demonstrate the same ability as reliable transfer.

Is system design necessary for every role?

Not in the same depth. Confirm level-specific expectations. Learn basic APIs, persistence, and failure handling regardless, but do not replace essential coding practice with advanced distributed-system trivia when the role does not require it.

How should I recover from a difficult interview?

Capture your own reasoning gaps without publishing confidential questions. Separate a knowledge gap from a communication or execution problem, choose a targeted correction, and test it on a new problem before the next round.

Avoid these traps

- Assuming Google, Meta, Amazon, Apple, and Netflix run identical loops.
- Memorizing anecdotes instead of understanding invariants and trade-offs.
- Treating a preparation plan as an offer or placement guarantee.

Ready for the next step?

- Mixed coding mocks are independently correct and tested.
- Design assumptions and failure boundaries are explicit.
- Behavioral stories reflect real personal contributions.
- The final plan matches the actual role and recruiter guidance.

Reliable next reads

- [Amazon SDE II interview preparation](#)
- [Google hiring overview](#)
- [Microsoft technical interviewing](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.